

MEDIATEK

MT8668 Yocto Panel User Manual

Version: 1.0
Release date: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者
1.0	2026-01-28	涂红云

目录

版本记录	2
目录 3	
1 概述	5
2 点屏前置条件	6
2.1 从屏厂或者串行器厂商获取如下信息	6
2.2 原理图 review	6
3 新增屏驱	7
3.1 Device 的修改	7
3.2 LK2 的修改	7
3.3 Kernel 的修改	7
4 重要事项	9
5 LK drivers	10
5.1 增加 LK2 屏驱	10
5.2 编写驱动	10
6 Kernel DRM drivers	17
6.1 增加 kernel 驱动	17
6.2 编写驱动	17
7 精确的 FPS 设置	21
8 Debug sop	22
8.1 Lk 阶段 debug	22
8.2 Kernel 阶段 debug	24
8.3 Resume 阶段 debug	24
8.4 花屏 debug	24
8.5 含有 bridge 屏的 debug	26
9 Dump reg	27
9.1 LK 阶段	27
9.2 Kernel Dump reg	27
9.3 adb 命令 dump reg	28
9.4 Test Pattern 命令	28
9.5 分析 reg	29
10 使用 MTK 通用 serdes driver	31
10.1 驱动中命令格式说明	31

10.2 DSI Ser 设定示例.....	32
10.3 DSI Des-Panel 设定示例.....	33
10.4 DSI Superframe 设定示例.....	33
10.5 DSI Dual Link 设定示例.....	35
10.6 Des Panel 兼容设定示例.....	37
10.7 使用注意.....	37
附件一 附加条款.....	38

1 概述

本文主要介绍 panel 驱动移植 SOP 以及相应的 debug 技巧

2 点屏前置条件

2.1 从屏厂或者串行器厂商获取如下信息

- 屏厂要在治具上使用外部输入信号点亮屏
- 点屏前确定“屏的上下电时序、几个 DSI port、几个 serdes、cmd/vdo mode、Cphy/dphy、Lane number 等信息”
- Init/deinit cmd（一定要是第一点提到的 cmd，即“治具上使用外部输入信号点亮屏”）
- 屏的 timing 信息，如 Width/height/vfp/vbp/vsa/hfp/hbp/hsa/fps/mipi clock 等等
- serdes test pattern（可选，点不亮时要求提供）
- Dsc 参数，几个 slice（如果要开 dsc，请先根据参考初步判断一下 MTK IC 是否能支持）
- 其他功能如何实现（backlight/fps change/cabc...）
- 如果接串行器，串行器要如何设置等等

2.2 原理图 review

- 屏的电源如何控制
 - 用我司 regulator 请找 PMIC owner 要如何控制（LK、kernel 配置都要）
 - 用 3rd IC 请找 3rd 要
- GPIO 如何配置
 - 拿到 GPIO 功能表，Dws 先配置好
- 背光如何配置
 - Bring up 时可以直接控制 GPIO 拉高实现

3 新增屏驱

3.1 Device 的修改

1. 在 ko_order_table 中加入 ko 名称，注意按驱动加载顺序写
2. 在 projectconfig.mk 中修改符合屏分辨率的 bootlogo 设定

3.2 LK2 的修改

注意：如果没有 LK，可以忽略此步骤

1. 添加新的驱动文件

```
M dev/lcm/include/lcm_drv.h
M dev/lcm/mt65xx_lcm_list.c
A dev/lcm/serdes_dsi_vdo/rules.mk
A dev/lcm/serdes_dsi_vdo/serdes_dsi_vdo.c
```

2. 在 \$(project).mk 中加入此驱动支持

```
:/quark/lk2/+/11291443/2/project/auto8668p1_64.mk
```

```
22
23 MODULE_DEPS += app/$(BOOT_APP)
24 MTK_LCM_LIST_SUPPORT="serdes_dsi_vdo jd9265da_wxga_dsi_vdo"
25 MTK_LCM_COMMON_DRIVER_SUPPORT := no
26
```

3. lcm list 中加入此驱动

```
/c/quark/lk2/+/11235614/5/dev/lcm/mt65xx_lcm_list.c
```

```
3/ #if defined(VIDH6126A_VDO)
38 &vtdr6126a_vdo_lcm_drv,
39 #endif
40
41 #if defined(SERDES_DSI_VDO)
42 &serdes_dsi_vdo_lcm_drv,
43 #endif
44
45 #if defined(MAX96789_DSI_VDO)
46 &max96789_dsi_vdo_lcm_drv,
47 #endif
```

4. lcm_drv.h 中加入此驱动

```
/lk2/+/11235614/5/dev/lcm/include/lcm_drv.h
```

```
1135 extern LCM_DRIVER nt36532_wxga2944_dsi_vdo_144hz_tianma_lc
1136 extern LCM_DRIVER hx83102j_fhdp_dsi_vdo_120hz_boe_dphy_lcm_
1137 extern LCM_DRIVER hx83102j_fhdp_dsi_vdo_120hz_tianma_dphy_1
1138 extern LCM_DRIVER vtdr6126a_vdo_lcm_drv;
1139 extern LCM_DRIVER serdes_dsi_vdo_lcm_drv;
1140
```

3.3 Kernel 的修改

注意：如果是 kernel-6.6 一样的修改，只是路径不一样

1. 添加新的驱动

```
M drivers/gpu/drm/mediatek/mediatek_v2/mtk_panel_ext.h
A drivers/gpu/drm/panel/bridge-serdes.h
A drivers/gpu/drm/panel/bridge-serdes.c
M drivers/gpu/drm/panel/BUILD.bazel
M drivers/gpu/drm/panel/Kconfig
```

2. 在 Kconfig 中加入新的驱动描述

```
/quark/kernel_device_modules-6.12/+11235586/7/drivers/gpu/drm/panel/Kconfig
63 »      vdo mode panel. The panel has a 1920x1080
64 »      resolution and uses 24-bits RGB per pixel. It provides a
65 »      MIPI DSI interface to host.
66
67 config DRM_BRIDGE_SERDES
68 »      tristate "serdes bridge vdo mode"
69 »      depends on OF
70 »      depends on DRM_MIPI_DSI
71 »      depends on BACKLIGHT_CLASS_DEVICE
72 »      help
73 »      Say Y here if you want to enable support for Serdes
74 »      vdo mode bridge. The bridge has a 1920x1080
75 »      resolution and uses 24-bits RGB per pixel. It provides a
76 »      MIPI DSI interface to host.
77
```

3. 在 BUILD.bazel 中加入新的驱动

```
/quark/kernel_device_modules-6.12/+11235586/7/drivers/gpu/drm/panel/BUILD.bazel
103 "panel-nt37701a-cmd-mipichg",
104 "panel-truly-td4330-cmd",
105 "panel-samsung-s68fc01-vdo-aod",
106 "panel-hx-nt37701-dphy-cmd",
107 "panel-hx-nt37701-dphy-cmd-120hz",
108 "panel-vtdr6126a-vdo",
109 "bridge-serdes",
110 ]
```

4. 在 kernel config 中加入驱动

```
/quark/kernel_device_modules-6.12/+11235588/2/arch/arm64/configs/mgk_64_k612_defconfig
259 CONFIG_MTK_VMM_NOTIFICATION_MT6993=m
260 CONFIG_MTK_PWMIC_PROTECT=m
261 CONFIG_DRM_PANEL_SERDES=m
262 CONFIG_DRM_BRIDGE_SERDES=m
263 CONFIG_DRM_PANEL_BOE_JD9365DA_VDO=m
264 CONFIG_DRM_PANEL_TRULY_TD4330_VDO=m
```

5. 在 kleaf 中加入驱动

```
/quark/kernel_device_modules-6.12/+11235589/2/kernel/kleaf/mgk_64.bzl#
795 "//kernel_device_modules-{}drivers/misc/mediatek/aee/mrdump:mrdump".format(kernel_version),
796 "//kernel_device_modules-{}drivers/misc/mediatek/gate_ic:rt4831a_drv".format(kernel_version),
797 "//kernel_device_modules-{}drivers/gpu/drm/panel:bridge-serdes".format(kernel_version),
798 "//kernel_device_modules-{}drivers/gpu/drm/panel:ktz8866".format(kernel_version),
799 "//kernel_device_modules-{}drivers/auo/drm/panel:panel-aw37501-i2c".format(kernel_version),
```

6. 在 dts 中加入驱动描述并在\$(project).dts 中 include (或者修改已存在的设定)

Commit message

```
A arch/arm64/boot/dts/mediatek/cust_mt8668_display_config_main.dtsi
A arch/arm64/boot/dts/mediatek/cust_mt8668_display_interface.dtsi
A arch/arm64/boot/dts/mediatek/cust_mt8668_display_serdes_config.dtsi
```

4 重要事项

- 为了防止从 LK 转 kernel 画面抖动，kernel 启动后 dsi/mipi_tx/panel 的设置都是用 LK 的参数，suspend/resume 之后才是全部采用 kernel 的配置。如果发现 suspend/resume 之后有参数跟上电启动不同，请检查这三部分的设置区别
- Kernel 启动时会获取一些参数给 display 使用，在 kernel panel driver 没有 ready 的情况下系统不能正常工作是正常现象
- 后面页面中写的 must set 的参数是必须要写的，其他部分根据需要填写

5 LK drivers

- LK2 路径:/vendor/mediatek/proprietary/bootable/bootloader/lk2/

Name	Path
Display Core	platform /mediatek/ mt6xxx /disp
Panel driver	dev / lcm /
LOGO picture	LK2: Vendor/mediatek/proprietary/external/BootLogo/logo LK: vendor/mediatek/proprietary/bootable/bootloader/lk/dev/logo

5.1 增加 LK2 屏驱

参考第 3 章新增 LK 屏驱文件和修改

5.2 编写驱动

1. 编写 lcm_drv 结构体(must set)

以<tv101wum_n16_wuxga_dsi_video_boe_rtq6752.c> 为例:

```
LCM_DRIVER tv101wum_n16_wuxga_dsi_vdo_boe_rtq6752_drv = {
    .name = "tv101wum_n16_wuxga_dsi_vdo_boe_rtq6752_drv",
    .set_util_funcs = lcm_set_util_funcs,
    .get_params = lcm_get_params,
    .init = lcm_init,
    .suspend = lcm_suspend,
    .resume = lcm_resume,
    .init_power = lcm_init_power,
    .compare_id = lcm_compare_id,
    .resume_power = lcm_resume_power,
    .suspend_power = lcm_suspend_power,
    .ata_check = lcm_ata_check,
    .update = lcm_update,
};
```

2. 填充 LCM 参数(must set)

按照实际参数参考其他屏驱填写

- Lcm 宽和高(must set)

```
static void lcm_get_params(LCM_PARAMS *params)
{
    memset(params, 0, sizeof(LCM_PARAMS));

    params->type = LCM_TYPE_DSI;

    params->width = FRAME_WIDTH;
    params->height = FRAME_HEIGHT;
```

- Lcm 参数 (dsi 设置, must set)

```

params->dsi.mode = SYNC_PULSE_VDO_MODE;
params->dsi.switch_mode = CMD_MODE;
params->dsi.switch_mode_enable = 0;

/* DSI */
/* Command mode setting */
params->dsi.LANE_NUM = LCM_FOUR_LANE;
/* The following defined the format for data coming from LCD engine.
params->dsi.data_format.color_order = LCM_COLOR_ORDER_RGB;
params->dsi.data_format.trans_seq = LCM_DSI_TRANS_SEQ_MSB_FIRST;
params->dsi.data_format.padding = LCM_DSI_PADDING_ON_LSB;
params->dsi.data_format.format = LCM_DSI_FORMAT_RGB888;

/* Highly depends on LCD driver capability. */
params->dsi.packet_size = 256;

```

- Lcm 参数(vdo timing, must set)

```

params->dsi.vertical_sync_active = VSA;
params->dsi.vertical_backporch = VBP;
params->dsi.vertical_frontporch = VFP;
params->dsi.vertical_frontporch_for_low_power = 750;
params->dsi.vertical_active_line = FRAME_HEIGHT;

params->dsi.horizontal_sync_active = HSA;
params->dsi.horizontal_backporch = HBP;
params->dsi.horizontal_frontporch = HFP;
params->dsi.horizontal_active_pixel = FRAME_WIDTH;
params->dsi.ssc_disable = 1;

```

- esd 配置(optional, 使用 bridge 必须关掉)

```

params->dsi.esd_check_enable = 1;
params->dsi.customization_esd_check_enable = 0;
params->dsi.lcm_esd_check_table[0].cmd = 0x0A;
params->dsi.lcm_esd_check_table[0].count = 1;
params->dsi.lcm_esd_check_table[0].para_list[0] = 0x9C;

```

- pll config (PLL_CLOCK 和 data_rate 至少要设置 1 个, 参考下面公式计算)

```

00179: #if.(LCM_DSI_CMD_MODE)
00180: ....params->dsi.PLL_CLOCK==551;
00181: #else
00182: ....params->dsi.PLL_CLOCK==270;
00183: #endif
00184: ....params->dsi.data_rate==563;

```

(DPHY) $\text{Data Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Blanking Ratio} \times \text{Bit Per Pixel}}{\text{Lane Num}}$

(DPHY DSC) $\text{Data Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Compression Ratio} \times \text{Bit Per Pixel} + \text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times (\text{Blanking Ratio} - 1) \times \text{Bit Per Pixel}}{\text{Lane Num}}$

(CPHY) $\text{Symbol Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Blanking Ratio} \times \text{Bit Per Pixel} \times 7/16}{\text{Trio Num}}$

(CPHY DSC) $\text{Symbol Rate} = \frac{\text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times \text{Compression Ratio} \times \text{Bit Per Pixel} \times 7/16 + \text{Hactive} \times \text{Vactive} \times \text{Frame Rate} \times (\text{Blanking Ratio} - 1) \times \text{Bit Per Pixel} \times 7/16}{\text{Trio Num}}$

- 展频配置 (optional, 使用 bridge 必须关掉)

MIPI Clock Spread Spectrum:

```

params->dsi.ssc_disable = 0;
params->dsi.ssc_range = 4;

```

Ssc_disable: 0 → enable ssc, 1 → disable ssc

Ssc_range: 1~8, default value:0

- mipi clock 配置(optional)

Mipi clk mode setting:

- a. non-continuous clk mode:

```
params->dsi.cont_clock = 0;
params->dsi.clk_lp_per_line_enable = 1;
```

- b. continuous clk mode:

```
params->dsi.cont_clock = 1;
/* params->dsi.clk_lp_per_line_enable = 1; */
```

- cphy/dphy 配置(by case, cphy 设置为 1, dphy 设置为 0)

```
params->dsi.IsCphy = 1;
```

- dual port 设置 (by case)

```
params->lcm_if = LCM_INTERFACE_DSI0;
params->lcm_cmd_if = LCM_INTERFACE_DSI_DUAL;

params->dsi.mode = CMD_MODE;

params->dsi.output_mode = MTK_PANEL_DUAL_PORT;
params->dsi.ssc_disable = 1;
```

- Dsc 配置(by case)

- 如果要用 DSC, 必须设置 dsc_enable=1, 并设定 DSC 参数
- 是否需要支持 DSC 取决于屏, 且 DSC 参数需要屏厂提供
- MediaTek 特有参数 dsc_cfg:
 - 8bpc to 8bpp 设置为 0x22,
 - 10bpc to 8bpp 设置为 0x828
- 其他参数——对应到屏厂提供参数
- dsc 设定例子

```

params->dsi.dsc_enable = 1;
params->dsi.dsc_params.ver = 17;
params->dsi.dsc_params.slice_mode = 1;
params->dsi.dsc_params.rgb_swap = 0;
params->dsi.dsc_params.dsc_cfg = 34;
params->dsi.dsc_params.rct_on = 1;
params->dsi.dsc_params.bit_per_channel = 8;
params->dsi.dsc_params.dsc_line_buf_depth = 9;
params->dsi.dsc_params.bp_enable = 1;
params->dsi.dsc_params.bit_per_pixel = 128;
params->dsi.dsc_params.pic_height = 2400;
params->dsi.dsc_params.pic_width = 1080;
params->dsi.dsc_params.slice_height = 8;
params->dsi.dsc_params.slice_width = 540;
params->dsi.dsc_params.chunk_size = 540;
params->dsi.dsc_params.xmit_delay = 170;
params->dsi.dsc_params.dec_delay = 526;
params->dsi.dsc_params.scale_value = 32;
params->dsi.dsc_params.increment_interval = 43;
params->dsi.dsc_params.decrement_interval = 7;
params->dsi.dsc_params.line_bpg_offset = 12;
params->dsi.dsc_params.nfl_bpg_offset = 3511;
params->dsi.dsc_params.slice_bpg_offset = 3255;
params->dsi.dsc_params.initial_offset = 6144;
params->dsi.dsc_params.final_offset = 7072;
params->dsi.dsc_params.flatness_minqp = 3;
params->dsi.dsc_params.flatness_maxqp = 12;
params->dsi.dsc_params.rc_model_size = 8192;
params->dsi.dsc_params.rc_edge_factor = 6;
params->dsi.dsc_params.rc_quant_incr_limit0 = 11;
params->dsi.dsc_params.rc_quant_incr_limit1 = 11;
params->dsi.dsc_params.rc_tgt_offset_hi = 3;
params->dsi.dsc_params.rc_tgt_offset_lo = 3;

```

■ 厂商提供的 pps 参数例子 1

800 2560 slice 20	Dec	Hex
force_reg_dsc_version_major[3:0] =	1	1
force_reg_dsc_version_minor[3:0] =	1	1
force_reg_pps_identifier[7:0] =	0	00
force_reg_bits_per_component[3:0] =	8	8
force_reg_linebuf_depth[3:0] =	9	9
force_reg_block_pred_enable =	1	1
force_reg_convert_rgb =	1	1
force_reg_simple_422 =	0	0
force_reg_vbr_enable =	0	0
force_reg_bits_per_pixel[9:0] =	128	80
force_reg_pic_height[15:0] =	2560	0A00
force_reg_pic_width[15:0] =	800	0320
force_reg_slice_height[15:0] =	20	14
force_reg_slice_width[15:0] =	800	0320
force_reg_chunk_size[15:0] =	800	0320
force_reg_initial_xmit_delay[9:0] =	512	0200
force_reg_initial_dec_delay[15:0] =	657	0291
force_reg_initial_scale_value[5:0] =	32	0020

■ 厂商提供的 pps 参数例子 2

```

unsigned char pps[] = {0x11, 0x00, 0x00, 0xAB, 0x30, 0x80, 0x07, 0x54,
0x0B, 0xB8, 0x00, 0x14, 0x02, 0xEE, 0x02, 0xEE,
0x02, 0x00, 0x01, 0xBC, 0x00, 0x20, 0x00, 0x90,
0x00, 0x0A, 0x00, 0x06, 0x10, 0x00, 0x12, 0x4E,
0x18, 0x00, 0x10, 0xF0, 0x07, 0x10, 0x20, 0x00,
0x06, 0x0F, 0x0F, 0x33, 0x0E, 0x1C, 0x2A, 0x38,
0x46, 0x54, 0x62, 0x69, 0x77, 0x77, 0x79, 0x7B,
0x7D, 0x7E, 0x02, 0x20, 0x00, 0x2A, 0x40,
0x2A, 0xBE, 0x3A, 0xFC, 0x3A, 0xFA, 0x3A, 0xF8,
0x3B, 0x3B, 0x3B, 0x78, 0x3B, 0xB6, 0x4B, 0xB6,
0x4B, 0xF4, 0x4B, 0xF4, 0x6C, 0x34, 0x84, 0x74,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};

```

– GPIO 配置(参考 GPIO SOP)

3. 编写 LCM init power 函数(must set)

按照 HW 设计实际情况编写

```
static void lcm_init_power(void)
{
    SET RESET_PIN(0);
    MDELAY(100);
    if (lcm_util.set_gpio_lcd_enp_bias) {
        lcm_util.set_gpio_lcd_enp_bias(1);
        i2c_w8(0x06, 0, 0x3e, 0x00, 0x0f);
        i2c_w8(0x06, 0, 0x3e, 0x01, 0x0f);

        LCM_LOGI("[zsq] -----lcm_init_power-----\n");
    }
    else
        LCM_LOGI("[zsq] set_gpio_lcd_enp_bias not defined\n");
}
```

4. 编写 LCM init 函数(must set)

按照屏的时序编写:

```
5: static void lcm_init(void *fdt)
6: {
7:     dprintf(0, "%s +++\n", __func__);
8:     SET_RESET_PIN(1, fdt);
9:     MDELAY(5);
0:     SET_RESET_PIN(0, fdt);
1:     MDELAY(15);
2:     SET_RESET_PIN(1, fdt);
3:     MDELAY(60);
4:
5:     is_send_data = false;
6:     push_table_by_type(lcm_bkl_setting,
7:         sizeof(lcm_bkl_setting) / sizeof(struct cust_setting_table),
8:         1);
9:     #if LCM_USED_2PORT_4SLICE
0:     push_table_by_type(lcm_pps_setting,
1:         sizeof(lcm_pps_setting) / sizeof(struct cust_setting_table),
2:         1);
3:     MDELAY(20);
4:     push_table_by_type(lcm_4slice_setting,
5:         sizeof(lcm_4slice_setting) / sizeof(struct cust_setting_table),
6:         1);
7:     MDELAY(20);
8:     #endif
9:     push_table(lcm_initialization_setting,
0:         sizeof(lcm_initialization_setting) / sizeof(struct LCM_setting_table),
1:         1);
2:
3:     dprintf(0, "%s ---\n", __func__);
4: } ? end lcm_init ?
5:
```

5. 编写 LCM suspend/resume 函数(optional)

```
static void lcm_suspend(void)
{
    int ret;

    push_table(lcm_suspend_setting,
        sizeof(lcm_suspend_setting) / sizeof(struct LCM_setting_table), 1);

    SET_RESET_PIN(0);
    MDELAY(10);
}

static void lcm_resume(void)
{
    lcm_init();
}
```

6. 编写 LCM compare 函数(如果是多屏兼容才需要)

```

#define LCM_ID_NT36672C_JDI (0x41)

static unsigned int lcm_compare_id(void)
{
    unsigned int id = 0;
    unsigned char buffer[2];
    unsigned int array[16];

    LCM_LOGI("%S: enter\n", __func__);

    SET RESET PIN(1);
    MDELAY(10);
    SET RESET PIN(0);
    MDELAY(10);
    SET RESET PIN(1);
    MDELAY(10);

    array[0] = 0x00013700; /* read id return two byte,version and id */
    dsi_set_cmdq(array, 1, 1);

    read_reg_v2(0xDA, buffer, 1);
    id = buffer[0]; /* we only need ID */

    LCM_LOGI("%S,nt3672C_id_jdi=0x%08x\n", __func__, id);

    if (id == LCM_ID_NT36672C_JDI)
        return 1;
    else
        return 0;
} ? end lcm_compare_id ?

```

7. 编写背光函数(使用 lcm_pwm 才需要)

```

static void lcm_setbacklight(unsigned int level)
{
    LCM_LOGI("%S,nt35695 backlight: level = %d\n", __func__, level);

    bl_level[0].para_list[1] = level;

    push_table(bl_level, sizeof(bl_level) / sizeof(struct LCM_setting_table), 1);
}

```

8. 填充初始化参数(must set)

```

static struct LCM_setting_table lcm_suspend_setting[] = {
    {0x28, 0, {} },
    {0x10, 0, {} },
    {REGFLAG_DELAY, 120, {} },
    {0x4F, 1, {0x01} },
    {REGFLAG_DELAY, 120, {} }
};

static struct LCM_setting_table init_setting[] = {
    {0xFF, 1, {0x24} },
    {0xFB, 1, {0x01} },
    {0x2D, 1, {0x08} },
    {0xFF, 1, {0x24} }, /* Return To CMD1 */
    {0x6E, 1, {0x10} }, /* Return To CMD1 */
    {0xFB, 1, {0x01} }, /* Return To CMD1 */
    {0xFF, 1, {0x10} }, /* Return To CMD1 */

    {0xFF, 1, {0x10} }, /* Return To CMD1 */
    {REGFLAG_UDELAY, 1, {} },
#ifdef (LCM_DSI_CMD_MODE)
    {0xBB, 1, {0x10} }, /*CMD MODE*/
#else
    {0xBB, 1, {0x03} }, /*VDO MODE*/
#endif
    {0x3B, 5, {0x03, 0x0A, 0x0A, 0x0A, 0x0A} },
    {0x53, 1, {0x24} },
    {0x55, 1, {0x00} },
    {0x5E, 1, {0x00} },
    {0xFF, 1, {0x24} }, /* CMD2 Page4 Entrance */
    {REGFLAG_UDELAY, 1, {} },
    {0xFB, 1, {0x01} },

```

9. 参考其他驱动编写其余重要部分(must set)

```

#define GPIO_LED_BIAS_EN GPIO87
#define GPIO_LCD_RESET GPIO85
#define GPIO_LCM_ID0 GPIO109
#define GPIO_LCM_ID1 GPIO98
#define GPIO_LCD_BIAS_ENP GPIO82
#define GPIO_LCD_BIAS_ENN GPIO81
static const unsigned int BL_MIN_LEVEL = 20;
static LCM_UTIL_FUNCS lcm_util;
#define BIAS_I2C_ADDRESS (0x11)

#define SET_RESET_PIN(v, fdt) (lcm_util.set_reset_pin(v, fdt))
#define MDELAY(n) (lcm_util.mdelay(n))
#define UDELAY(n) (lcm_util.udelay(n))

/* ----- */
/* Local Functions */
/* ----- */

#define dsi_set_cmdq_V2(cmd, count, ppara, force_update) \
    lcm_util.dsi_set_cmdq_V2(cmd, count, ppara, force_update)
#define dsi_set_cmdq(pdata, queue_size, force_update) \
    lcm_util.dsi_set_cmdq(pdata, queue_size, force_update)
#define write_cmd(cmd) lcm_util.dsi_write_cmd(cmd)
#define write_regs(addr, pdata, byte_nums) \
    lcm_util.dsi_write_regs(addr, pdata, byte_nums)
#define read_reg(cmd) \
    lcm_util.dsi_dcs_read_lcm_reg(cmd)
#define read_reg_v2(cmd, buffer, buffer_size) \
    lcm_util.dsi_dcs_read_lcm_reg_v2(cmd, buffer, buffer_size)

/* ----- */
/* Local Constants */
/* ----- */
#define LCM_DSI_CMD_MODE 0
#define FRAME_WIDTH (1200)
#define FRAME_HEIGHT (2000)

#define REGFLAG_DELAY 0xFFFC
#define REGFLAG_UDELAY 0xFFFB
#define REGFLAG_END_OF_TABLE 0xFFFF
#define REGFLAG_RESET_LOW 0xFFFE
#define REGFLAG_RESET_HIGH 0xFFFF

static void lcm_set_gpio_output(unsigned int GPIO, unsigned int output)
{
    mt_set_gpio_mode(GPIO, GPIO_MODE_00);
    mt_set_gpio_dir(GPIO, GPIO_DIR_OUT);
    mt_set_gpio_out(GPIO, (output > 0) ? GPIO_OUT_ONE : GPIO_OUT_ZERO);
}

struct LCM_setting_table {
    unsigned int cmd;
    unsigned char count;
    unsigned char para_list[200];
};

static void push_table(struct LCM_setting_table *table, unsigned int count,
    unsigned char force_update)
{
    unsigned int i;
    unsigned int cmd;
    for (i = 0; i < count; i++) {
        cmd = table[i].cmd;
        switch (cmd) {
            case REGFLAG_DELAY:
                if (table[i].count <= 10)
                    MDELAY(table[i].count);
                else
                    MDELAY(table[i].count);
                break;
            case REGFLAG_UDELAY:
                UDELAY(table[i].count);
                break;
            case REGFLAG_END_OF_TABLE:
                break;
            default:
                dsi_set_cmdq_V2(cmd, table[i].count, table[i].para_list,
                    force_update);
                break;
        }
    }
} ? end for i=0;i<count;i++ ?
} ? end push_table ?

static void lcm_set_util_funcs(const LCM_UTIL_FUNCS *util)
{
    memcpy(&lcm_util, util, sizeof(LCM_UTIL_FUNCS));
}

/* ----- */
/* Local Functions */
/* ----- */

#define dsi_set_cmdq_V2(cmd, count, ppara, force_update) \
    lcm_util.dsi_set_cmdq_V2(cmd, count, ppara, force_update)
#define dsi_set_cmdq(pdata, queue_size, force_update) \
    lcm_util.dsi_set_cmdq(pdata, queue_size, force_update)
#define write_cmd(cmd) lcm_util.dsi_write_cmd(cmd)
#define write_regs(addr, pdata, byte_nums) \
    lcm_util.dsi_write_regs(addr, pdata, byte_nums)
#define read_reg(cmd) \
    lcm_util.dsi_dcs_read_lcm_reg(cmd)
#define read_reg_v2(cmd, buffer, buffer_size) \
    lcm_util.dsi_dcs_read_lcm_reg_v2(cmd, buffer, buffer_size)

/* ----- */
/* Local Constants */
/* ----- */
#define LCM_DSI_CMD_MODE 0
#define FRAME_WIDTH (1200)
#define FRAME_HEIGHT (2000)

#define REGFLAG_DELAY 0xFFFC
#define REGFLAG_UDELAY 0xFFFB
#define REGFLAG_END_OF_TABLE 0xFFFF
#define REGFLAG_RESET_LOW 0xFFFE
#define REGFLAG_RESET_HIGH 0xFFFF

```

6 Kernel DRM drivers

- 文件路径: /kernel/ kernel_device_modules-6.12/drivers/gpu/drm/mediatek/Mediatek_v2

Name	Files
DRM framework relative	mtk_drm_drv.c, mtk_drm_crtc.c, mtk_drm_fb.c,mtk_drm_gem.c, mtk_drm_plane.c...
MTK DDP	mtk_disp_ccorr.c , mtk_disp_gamma.c, mtk_disp_dither.c,mtk_disp_ovl.c,mtk_disp_rdma.c, mtk_disp_dsi.c , mtk_disp_wdma.c ...
Connector	mtk_dsi.c,mtk_dpi.c,mtk_hdmi.c,mtk_mipi_tx.c...

Name	Path
DRM Core	drivers/gpu/drm/
MTK implement	drivers/gpu/drm/mediatek/mediatek_v2 (K510) OR drivers/gpu/drm/mediatek/ (K419)
Panel driver	drivers/gpu/drm/panel/
DRM Header File	include/drm/ include/uapi/drm/

6.1 增加 kernel 驱动

参考第 3.3 章新增修改 kernel 驱动

6.2 编写驱动

1. 填充 kernel 驱动结构体(must set)

- DRM 原始结构体

```
static const struct drm_display_mode default_mode = {  
    .clock = 422163, // Pixel clock  
    .hdisplay = 1080, // Frame Width  
    .hsync_start = 1080 + 274, // HFP  
    .hsync_end = 1080 + 274 + 12, // HSA  
    .htotal = 1080 + 274 + 12 + 56, // HBP  
    .vdisplay = 2400, // Frame Height  
    .vsync_start = 2400 + 2528, // VFP  
    .vsync_end = 2400 + 2528 + 10, // VSA  
    .vtotal = 2400 + 2528 + 10 + 10, // VBP  
};
```

2. 编写 MTK 扩展结构体(must set)

```
static struct mtk_panel_params ext_params = {
    .pll_clk = 369,
    .data_rate = 738,
    .vfp_low_power = 4178,
    .cust_esd_check = 0,
    .esd_check_enable = 1,
    .lcm_esd_check_table[0] = {
        .cmd = 0x0A, .count = 1, .para_list[0] = 0x9C,
    },
    .is_cphy = 1,
};
```

- Superframe 模式下, pll_clk/crop_width/crop_height/physical_width/physical_height 必须设置, 其他情况下 pll_clk 必须设置.
- Pll_clk: mipi clock
- Crop width/crop height: super frame 模式下单个屏的宽高.
- Physical_width/physical_height: super frame 模式下, physical_width 为两个屏宽相加, physical_height 为两个屏中高比较大的值.

3. 填充 Drm 架构驱动函数(must set):

- DRM 原始架构函数

```
static const struct drm_panel_funcs jdi_drm_funcs = {
    .disable = jdi_disable,
    .unprepare = jdi_unprepare,
    .prepare = jdi_prepare,
    .enable = jdi_enable,
    .get_modes = jdi_get_modes,
};
```

Diagram illustrating the mapping of DRM panel functions to their corresponding actions:

- `.disable` → Panel power disable
- `.unprepare` → Panel suspend sequence
- `.prepare` → Panel power on and perform set up
- `.enable` → Enable Panel
- `.get_modes` → Panel mode configuration

4. 编写 probe 函数(Must set)

```
static int lcm_probe(struct mipi_dsi_device *dsi)
{
    struct device *dev = &dsi->dev;
    struct device_node *dsi_node, *remote_node = NULL, *endpoint = NULL, *ser_node;
    const phandle *ser_phandle;
    struct lcm *ctx;
    int ret;
    const struct panel_desc *desc;

    pr_info("%s+, name:%s\n", __func__, dev->of_node->name);

    dsi_node = of_get_parent(dev->of_node);
    if (dsi_node) {
        endpoint = of_graph_get_next_endpoint(dsi_node, NULL);
        if (endpoint) {
            remote_node = of_graph_get_remote_port_parent(endpoint);
            if (!remote_node) {
                pr_info("No panel connected, skip probe lcm\n");
                return -ENODEV;
            }
            pr_info("device node name:%s\n", remote_node->name);
        }
    }
    if (remote_node != dev->of_node) {
        pr_info("%s+ skip probe due to not current lcm\n", __func__);
        return -ENODEV;
    }
}
```

5. 编写 remove 函数(Must set)

```

}; static void lcm_remove(struct mipi_dsi_device *dsi)
{
    struct lcm *ctx = mipi_dsi_get_drvdata(dsi);
    #if defined(CONFIG_MTK_PANEL_EXT)
    struct mtk_panel_ctx *ext_ctx = find_panel_ctx(&ctx->panel);
    #endif

    mipi_dsi_detach(dsi);
    drm_panel_remove(&ctx->panel);
    #if defined(CONFIG_MTK_PANEL_EXT)
    mtk_panel_detach(ext_ctx);
    mtk_panel_remove(ext_ctx);
    #endif
}

```

6. 编写 prepare 函数(Must set)

```

static int lcm_prepare(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    lcm_poweron(ctx);
    lcm_panel_init(ctx);
    serdes_pre_enable(ctx->bridge);

    #if defined(CONFIG_MTK_PANEL_EXT)
    mtk_panel_tch_rst(panel);
    #endif
    pr_info("%s -\n", __func__);

    return 0;
}

```

7. 编写 init 函数(Must set)

```

static void lcm_panel_init(struct lcm *ctx)
{
    pr_info("%s+\n", __func__);
    pr_info("%s-\n", __func__);
}

```

8. 编写 enable 函数(Must set)

```

static int lcm_enable(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);

    pr_info("%s +\n", __func__);

    serdes_enable(ctx->bridge);

    pr_info("%s -\n", __func__);

    return 0;
}

```

9. 编写 disable 函数(Must set)

```
static int lcm_disable(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);
    pr_info("%s +\n", __func__);
    serdes_disable(ctx->bridge);
    pr_info("%s -\n", __func__);
    return 0;
}
```

10. 编写 unprepare 函数(Must set)

```
static int lcm_unprepare(struct drm_panel *panel)
{
    struct lcm *ctx = panel_to_lcm(panel);
    pr_info("%s +\n", __func__);
    lcm_poweroff(ctx);
    pr_info("%s -\n", __func__);
    return 0;
}
```

11. 编写 get_modes 函数(must set)

```
static int lcm_get_modes(struct drm_panel *panel,
                        struct drm_connector *connector)
{
    struct drm_display_mode *mode;

    pr_info("%s +\n", __func__);
    mode = drm_mode_duplicate(connector->dev, &default_mode);
    if (!mode) {
        pr_info("failed to add mode %ux%ux@%u\n",
                default_mode.hdisplay,
                default_mode.vdisplay,
                drm_mode_vrefresh(&default_mode));
        return -ENOMEM;
    }

    mode->type = DRM_MODE_TYPE_DRIVER | DRM_MODE_TYPE_PREFERRED;

    drm_mode_set_name(mode);
    drm_mode_probed_add(connector, mode);

    connector->display_info.width_mm = default_mode.width_mm;
    connector->display_info.height_mm = default_mode.height_mm;

    pr_info("%s -\n", __func__);

    return 1;
} « end lcm_get_modes »
```

12. 编写 MTK 扩展函数 reset (optional)

13. 编写 MTK 扩展函数 set_backlight_cmdq function(by case)

14. 编写 MTK 扩展函数 ext_param_set function(by case)

15. 编写 MTK 扩展函数 ata_check function(optional)

7 精确的 FPS 设置

- 手写笔应用中，需要 FPS 非常精准（120+-0.01）通过 data_rate_khz 来实现
- 首先通过表格(找 PM 要)计算出近似值（ideal bit freq）填到 data_rate，再通过 data_rate_khz 进行微调
- 注意即使设置了 data_rate_khz，data_rate/pll_clk 也需要设置（不管是 LK 还是 kernel 都是）
- Data_rate_khz 需要微调后实测 FPS，目前不能通过理论计算
- vdo_keep_hs_perline 对应下面表格 video mode keep hs mode，此项对 vdo mode fps 影响比较大

輸入參數	不可動	可自行填入						
1 DSI Frame Size		Vertical (unit: line)				Horizontal (unit: pixel) 只有HFP可調整frame rate，其他都按照客戶需求		
H_Total (pixel)	V_Total (line)	VACT	VFP	VPW	VBP	HACT	HFP	HSA
719	1962	1840	26	2	94	1472	110	10
Option							Real Horizontal (unit: pixel)	
DSI_BLLP_WC	Video Mode Keep HS Mode	Command Mode Keep HS Mode	Null Packet Word Count (Command Mode)	PROGSEQ_SKIP_EN	PHY Timing Post Processing	Display Pipe	HFP	HSA
10	Y	HS mode w/ Null Packet	13	0	1	1	113.33	10.00

需求	0: D-PHY 1: C-PHY	D-PHY: 1~4 C-PHY: 1~3	Compression Ratio	24: RGB888 30: RGB101010	range: 0~3	Bit Rate (D-PHY) Symbol Rate (C-PHY)
HBP	C/D-PHY	lane_num	DSC	PixelBit (bit/pixel)	mode selector	Ideal Bit_Freq
108	0	4	0.3333	24	1	1226

el)	PHY Timing Spec. (unit: UI)		Unit: Frame per Second(FPS)	Line Time Unit: us	Frame Time Unit: ms	Unit: MHz
HBP	t3-prebegin	t3-post	Real Frame Rate (FPS)	Real LineTime	Real FrameTime	Disp Clock
108.67	0	0	144.1127	3.537	6.939	457.8273063

```

00397: },
00398:
00399: struct mtk_panel_params {
00400:     unsigned int change_fps_by_vfp_send_cmd;
00401:     unsigned int pll_clk;
00402:     unsigned int data_rate;
00403:     /* t3-pre, t3-post, t3-post, t3-post, t3-post, event if data_rate_khz is set
00404:     unsigned int data_rate_khz; //only used in exact value for fps
00405:     unsigned int vdo_per_frame_lp_enable; /* Enable video mode per frame lp */
00406: };

```

```

00855: .data_rate_khz = 960298,
00856: .data_rate = 960,
00857: .data_rate_khz = 960,

```

8 Debug sop

8.1 LK 阶段 debug

LK 屏不亮

- 背光是否亮？
 - 是，继续下面步骤
 - 否，检测背光控制方式是否正确（LK 也会采用 dts 里面的设置）
 - 也可采用直接拉高 GPIO 的方式先避开
 - 如果使用背光 IC，询问对应 vendor 要如何控制 disp_pwm mode
 - 背光控制模式是否正确
 - Disp_pwm 模式

```

pwmleds: pwmleds {
    compatible = "mediatek,pwm-leds";
    status = "disabled";
    backlight {
        label = "lcd-backlight";
        pwms = <&disp_pwm0 0 500000>;
        max-brightness = <255>;
        pwm-names = "lcd-backlight";
    };
};

```

- I2c 模式

```

mtk_leds: mtk_leds {
    backlight {
        label = "lcd-backlight";
        max-brightness = <255>;
        max-hw-brightness = <2047>;
    };
};

mtk_leds {
    compatible = "mediatek,i2c-leds";

    backlight {
        led_mode = <6>;
        gate_enable = <1>;
        pwm_config = <0 1 0 0 0>;
    };
};

```

- Lcm pwm mode

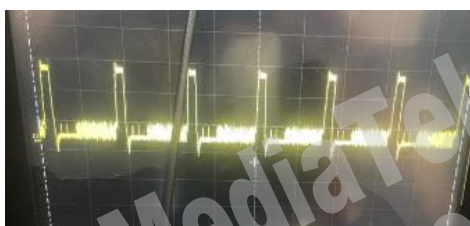
```

1:
2: &mtk_leds {
3:     compatible = "mediatek,disp-leds";
4:
5:     backlight {
6:         led_mode = <4>;
7:         gate_enable = <1>;
8:         pwm_config = <0 1 0 0 0>;
9:     };
0: };

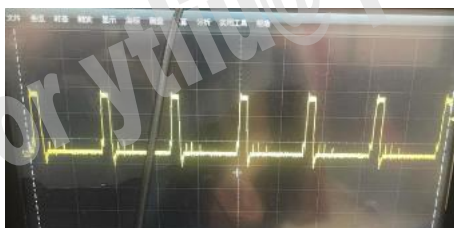
```

- 屏所需要的电压是否有正常供上？
 - 是，继续下面步骤
 - 否，用示波器量各路电压，看哪一路不正常，询问 PMIC owner 要如何控制

- 上电时序是否符合屏的要求
- 用示波器量信号
- Init cmd 是否有发成功? (判断标志, 读屏的 0x0A 为非 0 值, 需屏支持)
 - 是, 继续下面步骤
 - 否
 - 发送 cmd 是用 generic? 还是 dcs? 还是混用? 默认 cmd<0xb0 用 dcs, cmd>=0xb0 用 generic
 - 找屏厂要屏的 BIST pattern 命令, 看 BIST 是否能正常出来
 - Bist 能亮说明屏的供电没问题
 - Bist 也不亮
 - 是否有 lane swap? 量 D0P/N 是否有波形出来
 - init cmd 是否给错, 用逻辑分析仪抓发送 init cmd 的数据给屏厂检查
 - 如果使用 bridge, 检测 bridge 的状态寄存器提供信息给 bridge 厂商分析
- Mipi 信号是否正常(做 mipi cts 测试, 需要所有项都 pass)
 - 有 vdo 数据的 Dphy 波形



- 没有 vdo 数据的 Dphy 波形



- Cphy 波形



- 是否有放 logo 文件? 若无 logo 文件, 屏上只会显示一行小字, 需仔细观察

8.2 Kernel 阶段 debug

进 kernel 后屏不亮：

- 量屏的各路电压是否有掉？
 - 进 kernel 后 regulator 在没人 enable 会自动关掉，屏驱中需要获取资源并 enable 它
- 背光是否有亮？
 - AAL 可能会导致背光关闭，关闭 AAL 看看是否有问题
 - 调整背光的函数是否能跑到，如不能需要加 log debug
- Kernel 屏驱是否有正常加载？
 - 观察 log，看驱动加载是否正常。Kernel 运行过程中需要一些参数
- 是否有使能 esd，esd 也会导致黑屏
- 是否有多种 mode 切换？Bring up 阶段先点一个 mode

8.3 Resume 阶段 debug

Resume 后屏不亮：

- 量屏的各路电压是否有正常起来
- 检查屏的 power on code 是否有异常
- 背光是否有亮？
 - 控制背光 code 是否有异常
- Kernel 屏驱是否有正常加载？
 - 观察 log，看驱动加载是否正常。
- 对比跟 LK 的驱动，看两边设置是否一样

8.4 花屏 debug

是否有开 DSC

- 是，检测 DSC 设置
 - 打 DSC test pattern 是否能显示正常

```
echo reg_write 0x14015078 0x400000FF > /proc/clkgdbg ; cat /proc/clkgdbg
```

0x14015xxx 是 dsc base reg，根据 SOC 不同而不同，请询问 MediaTek 工程师正确的值

0x078 是 dsc test pattern reg 地址，根据 SOC 不同而不同，请询问 MediaTek 工程师正确的值

- 能，找 display owner 寻求帮助

- 不能，是否存在正常显示时刻，有正常时刻。dump dsc reg 对比是否有设置不一样
- Dump 命令：

```
adb shell "echo mobile:on > /d/mtkfb";adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb" > mtkfb.txt
```

搜索 DSC 部分

- 典型 dsc 花屏图片:



- 类似花屏是 AFBC 导致，使用如下命令关闭 AFBC 看看：

```
adb shell setprop debug.mediatek.disp_decompress 0
adb shell stop
adb shell start
```

- 典型 AFBC 花屏图片:



- 打 DSI test pattern 是否能正常显示

命令 (参考 dump reg) :

```
echo reg write 0x14017178 0x31 > /proc/clkgdbg ; cat /proc/clkgdbg
```

0x1401xxxx 是 dsi reg 地址，根据 SOC 不同而不同，请问 MediaTek 工程师

0x178 是 dsi test pattern 地址, 根据 SOC 不同而不同, 请问 MediaTek 工程师

- 能正常显示，找 display owner 寻求帮助
- 不能，log 中是否存在 `dsi underrun` 字样

- 是否有显示正常的情况
 - 是, dump 正常和不正常时的 DSI/MIPITX/DSC reg 值, 看设置是否有不一样
 - Dump 命令:

```
adb shell "echo mobile:on > /d/mtkfb";adb shell "echo diagnose>/d/mtkfb && cat /d/mtkfb" > mtkfb.txt
```
 - Dump 命令: 参考 dump reg 这一块, 用 adb cmd 的方法读取也可以
- 是否经过某种操作后才出现(比如切换 fps)
 - 对比切换前后 reg 是否一样
 - 例:切换 fps 后花屏
 - 询问屏厂切换 fps 是否存在限制, 是否要发送 cmd 等
 - Fps 切换时 mmclk 改变先后顺序是否正确 (mmclk 变高要先设 mmclk 后切 fps, mmclk 变低要先切 fps 后设置 mmclk, mmclk 计算方法参考 mtk_dsi_set_mmclk_by_datarate 函数, 切换逻辑参考 mtk_crtc_disp_mode_switch_begin 函数)

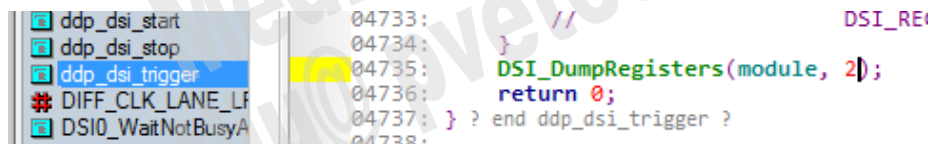
8.5 含有 bridge 屏的 debug

- 查看 bridge 状态寄存器, 可以得知当前 bridge 工作状态
- 从 bridge vendor 获取帮助

9 Dump reg

9.1 LK 阶段

- 在 Ddp_dsi.c/ddp_dsi_trigger() 最后调用 DSI_DumpRegisters()

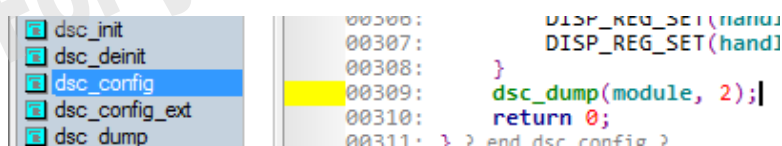


```

04733: // DSI_REI
04734: }
04735: DSI_DumpRegisters(module, 2);
04736: return 0;
04737: } ? end ddp_dsi_trigger ?
04738:

```

- 在 Ddp_dsc.c/dsc_config() 最后调用 dsc_dump()



```

00300: DISP_REG_SET(handl
00307: DISP_REG_SET(handl
00308: }
00309: dsc_dump(module, 2);
00310: return 0;
00311: } ? end dsc_config ?

```

- LK display log 默认没开，需要在 ddp_log.h 中打开

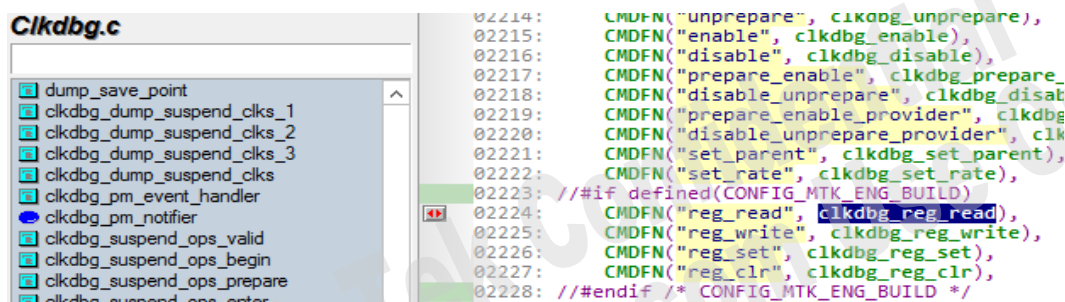
```

1:
2: #define DDP_INFO 0
3: #define DDP_CRITICAL 0
4: #define LOCAL_TRACE 1
5:

```

9.2 Kernel Dump reg

- 如何打开寄存器 adb debug 接口
 - 路径: kernel-xx/drivers/clk/mediatek/clktb.c/common_cmds[]
 - 函数: clktb_reg_read()和 clktb_reg_write()
 - 方法: 如果 clktb_reg_read() and clktb_reg_write()被#if defined(CONFIG_MTK_ENG_BUILD) 包起来了，请把#if defined(CONFIG_MTK_ENG_BUILD) mask 掉，同时把定义 clktb_reg_read() and clktb_reg_write()地方的#if defined(CONFIG_MTK_ENG_BUILD) 也 mask 掉，然后就可以用 adb command 改寄存器了



```

02214: CMDFN("unprepare", clktb_unprepare),
02215: CMDFN("enable", clktb_enable),
02216: CMDFN("disable", clktb_disable),
02217: CMDFN("prepare_enable", clktb_prepare_enable),
02218: CMDFN("disable_unprepare", clktb_disable_unprepare),
02219: CMDFN("prepare_enable_provider", clktb_prepare_enable_provider),
02220: CMDFN("disable_unprepare_provider", clktb_disable_unprepare_provider),
02221: CMDFN("set_parent", clktb_set_parent),
02222: CMDFN("set_rate", clktb_set_rate),
02223: // #if defined(CONFIG_MTK_ENG_BUILD)
02224: CMDFN("reg_read", clktb_reg_read),
02225: CMDFN("reg_write", clktb_reg_write),
02226: CMDFN("reg_set", clktb_reg_set),
02227: CMDFN("reg_clr", clktb_reg_clr),
02228: // #endif /* CONFIG_MTK_ENG_BUILD */

```

9.3 adb 命令 dump reg

- 寄存器读写命令

- 读:

```
echo reg_read addr> /proc/clkgdb ; cat /proc/clkgdb
```

- 按长度读:

```
echo reg_read_len addr length> /proc/clkgdb ; cat /proc/clkgdb
```

- 写:

```
echo reg_write addr value> /proc/clkgdb ; cat /proc/clkgdb
```

- 举例:

```
echo reg_read 0x1400D178 > /proc/clkgdb ; cat /proc/clkgdb
echo reg_write 0x1400D178 0x31 > /proc/clkgdb ; cat /proc/clkgdb
echo reg_read_len 0x1400D000 0x00001000 > /proc/clkgdb ; cat /proc/clkgdb
```

注意:

- 通常 MediaTek supporter 会说请把 xxx 的 yyy 写 zzz, 其中 xxx 代表某个模块, 通过前面 debug sop 章节 (130 页) 获取该模块的基地址, yyy 代表偏移量, zzz 代表 value

- 该语句最终需要执行的命令是:

```
echo reg_write xxx 基地址+yyy zzz> /proc/clkgdb ; cat /proc/clkgdb
```

- 例如调整 mipi 的驱动能力, supporter 会说把 mipi_tx 的 0x10 bit6-9 写 0111, 执行的命令是:

```
echo reg_read 0x11f60010 > /proc/clkgdb;cat /proc/clkgdb
```

- mipi_tx 基地址 0x11f60000, 先读 mipi_tx 的 0x10, 假如读到是 0, 将 0 的 bit6-9 或上 0111 得到 0x1c0

- 最终命令:

```
echo reg_write 0x11f60010 0x1c0> /proc/clkgdb ; cat /proc/clkgdb
```

9.4 Test Pattern 命令

- 使用 adb:

```
echo reg_write "DSI/DSC_base_reg+offset_addr" value> /proc/clkgdb ; cat /proc/clkgdb
```

- DSI/DSC 基地址参考前面的说明

- Offset:

- Dsi:0x178 or 0x17c
 - Dsc:0x78 or 0x6c, 向 MediaTek 工程师确认

- Value:

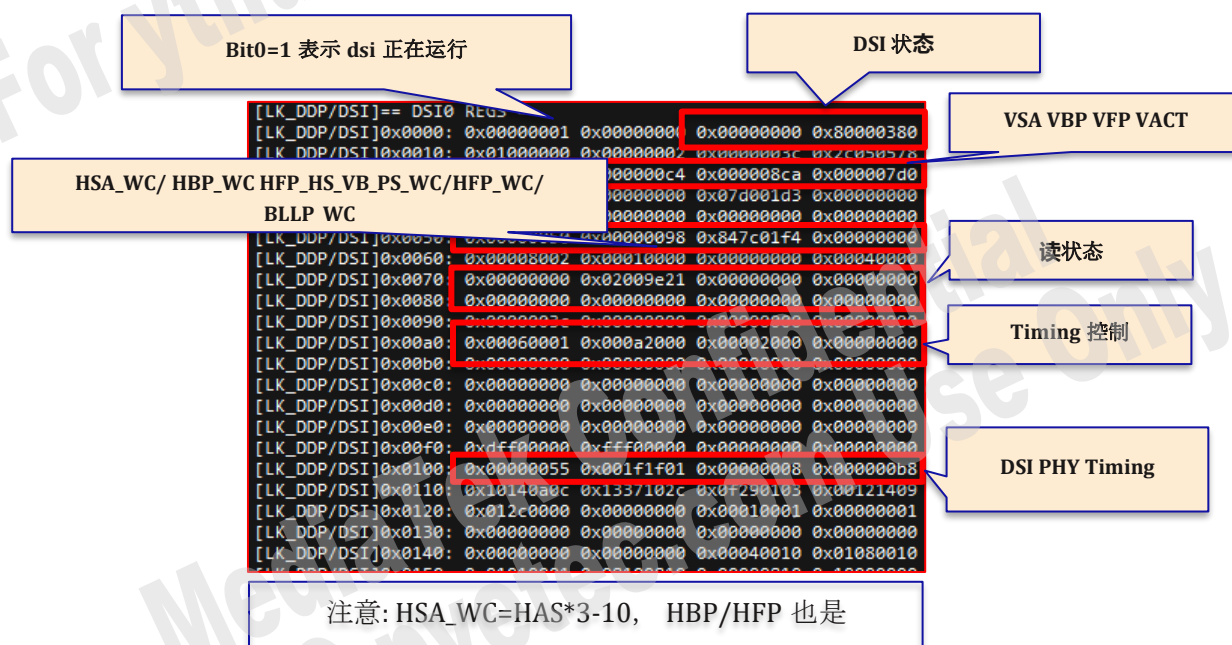
- Dsi: 0xc41
 - Dsc: 0x400000FF(blue), 向 MediaTek 工程师确认

- 在 code 中加入:

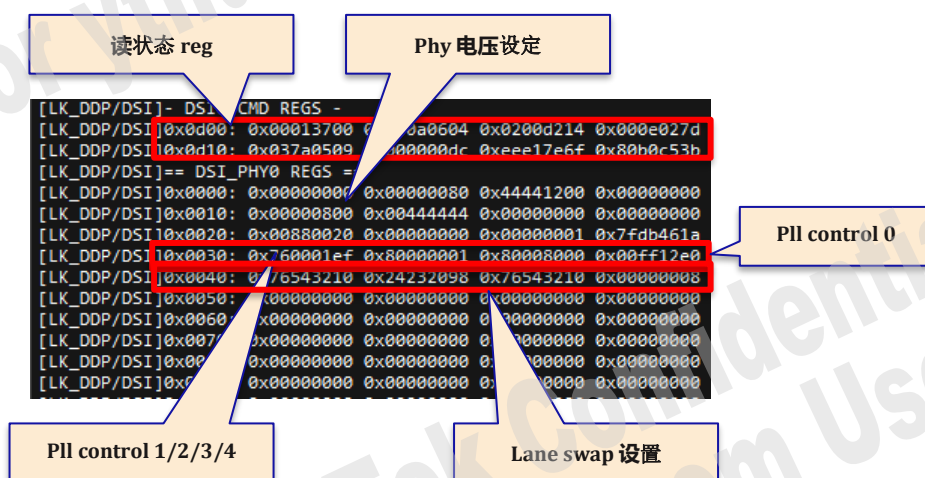
- DSI: 在 `Ddp_dsi_trigger` 函数最后调用 `DSI_OUTREG32(NULL, DSI_REG_BASE[0]+offset, value);`
- DSC: 在 `dsc_config` 函数最后调用 `DISP_REG_SET(handle, base+offset, value);`
- Offset/value 参考 `adb` 命令

9.5 分析 reg

- 如下 reg 说明, 仅供参考, 具体 reg 内容请参考 coda.
- DSI 向 8:



- MIPI TX



- DSC

DSC 控制寄存器		DSC 状态		DSC 宽		DSC 高	
DISP0	DSC REGS	addr	0x14015000				
014015000:	0x00010081	0x00000000	0x00000001	0x00000000	0x00000000	0x00000000	0x00000000
014015010:	0x00000000	0x00000000	0x01d20578	0x07cf07cf	0x00000000	0x00000000	0x00000000
014015020:	0x00e902bc	0x40630013	0x000002bc	0x000036b0	0x00000000	0x00000000	0x00000000
014015030:	0x00010001	0x00000022	0x00000001	0x057802bc	0x00000000	0x00000000	0x00000000
014015040:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015050:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015060:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015070:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015080:	0x000c8089	0x027d0200	0x020e0020	0x000d0009	0x00000000	0x00000000	0x00000000
014015090:	0x03dc057a	0x10d01800	0x2000c03	0x330b0b06	0x00000000	0x00000000	0x00000000
0140150a0:	0x382a1c0e	0x69625446	0x7b797770	0x0007e7d	0x00000000	0x00000000	0x00000000
0140150b0:	0x00800880	0xf8c100a1	0xe8e3f0e3	0xe103e0e3	0x00000000	0x00000000	0x00000000
0140150c0:	0xd943e123	0xd165d945	0xd189d165	0x000d1ac	0x00000000	0x00000000	0x00000000
0140150d0:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0140150e0:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0140150f0:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015100:	0x000002fc	0x0000041c	0x1000244a	0x10002648	0x00000000	0x00000000	0x00000000
014015110:	0x0c000000	0x00000000	0x00000000	0x000001e8	0x00000000	0x00000000	0x00000000
014015120:	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015130:	0x02a6429b	0x02a6429b	0x3fff0000	0x00006d60	0x00000000	0x00000000	0x00000000
014015140:	0x00e900e9	0x000007d0	0x00000001	0x00000000	0x00000000	0x00000000	0x00000000
014015150:	0x000e4070	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015160:	0xa02c2001	0xa02c2001	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
014015170:	0x00000000	0x11111111	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

10 使用 MediaTek 通用 serdes Driver

MediaTek 提供了一个 max96789+max96752 的通用驱动，可参考其写自己的驱动:

1. KO table 中加入 birdge-serdes.ko (如果不存在时)
2. LK 的 project.mk 中设置 `MTK_LCM_LIST_SUPPORT="serdes_dsi_vdo"`
3. kleaf 中加入 birdge-serdes.ko (如果不存在时)
4. defconfig 中加入 `CONFIG_DRM_BRIDGE_SERDES=m`(如果不存在时)
5. 如果需要使用 hotplug 功能，需要在 defconfig 中设置 `CONFIG_ENABLE_SERDES_HOTPLUG=y`
6. 如果 hotplug 功能要使用中断方式，需要在 code 中将 `ENABLE_HOTPLUG_INT` 定义为 1 (同时要修改 dws 将对于 GPIO 设置为 EINT 功能)
7. dts 中加入如第 10.2, 10.3, 0, 10.5 和 10.6 章的设定
8. 其他设定均在 driver 中 hard code 写死，如有不满足需求可以剥离出来写到 dts
9. 该驱动同时支持 dsi 和 dp/edp。
10. LK 中固定为 dsi0 对应的设定，如果要改为其他接口，需要改 code。
11. edp 的 init cmd 使用 work 队列去发送，会缩短发送时间但发送时序无法保证。如果不使用 work，设置 `ENABLE_INIT_WORK` 为 0。
12. 该驱动支持 des hotplug。如果不要支持 hotplug，设置 `CONFIG_ENABLE_SERDES_HOTPLUG` 为 n。
13. 如果没有 status command，不能支持 hotplug 和 mbrain 功能

10.1 驱动中命令格式说明

写命令格式 1	<i2c addr> <reg width> <reg addr> <data> <delay ms>
写命令格式 2	<i2c addr> <reg width> <cmd len> <data>
读命令格式	<i2c addr> <reg width> <reg addr> <mask> <except data>
i2c addr	要操作设备的 i2c client 地址
reg width	i2c 设备的 reg 地址长度，只支持 16/8/0 三种 16 表示 reg 地址是 2 个 byte 8 表示 reg 地址是 1 个 byte 0 表示不指定 reg 地址，后面全是数据
reg addr	要读写的 reg 的地址
data	要写到 reg 的数据

delay ms	发送完该命令后 delay 多少毫秒
cmd len	reg 地址长度为 0 时, 后面有多少 byte 的数据
mask	读出的数据跟这个数据做与操作
except data	读出的数据与 mask 做完与操作后预期的数据

10.2 DSI Ser 设定示例

```

&i2c1 {
    status = "okay";
    max96789: max96789@40 {
        compatible = "maxim,max96789,dsi0";          # compatible,如果是其他 chip 注意修改 driver, 注
        意红字表示该 driver 对应哪个接口
        status = "disabled";
        reg = <0x40>;
        reset-gpios = <&pio 37 0>;
        //power-en-gpios = <&pio 38 0>,<&pio 39 0>; # ser 电源开关, 按实际情况填写 (code 中只支持
        gpio 模式, 多个 GPIO 按上电顺序填写, 没有不写)
        //interrupt-parent = <&pio>;                  # 使用中断来做 hotplug, 如启用需在 driver 中
#define ENABLE_HOTPLUG_INT 1
        //interrupts = <187 IRQ_TYPE_EDGE_RISING>;
        pinctrl-names = "default";
        config = <&config_compatible>;                # 配置此 node 使用哪个 des panel 设置
        port {
            max96789_in: endpoint {
                remote-endpoint = <&dsi_out>;          # 标准的 bridge node 写法
            };
        };
    };
    max96789_v0: max96789-v0@4a {                    # 虚拟 ser driver, superframe or dual link
        时使用
        compatible = "maxim,max96789,virtual";
        status = "disabled";
        reg = <0x4a>;
        master = <&max96789>;                          # virtual ser driver 必须设置, 该设定对应到哪
        个实体设备
        port {
            max96789_v0_in: endpoint {
                remote-endpoint = <&dsiv0_out>;
            };
        };
    };
};

```

10.3 DSI Des-Panel 设定示例

```

huayang_1080: config1 {                                # single panel config node
    pre-init-cmd = <                                     # ser 初始化命令，在 bridge preenable 中执行
        0x40 0x10 0x0001 0x08 0x00
        0x40 0x10 0x0203 0x00 0x00
        .....
    >;
    post-init-cmd = <>;                                  # post ser 初始化命令，在 bridge pre_enable 中执行
    deinit-cmd = <                                       # ser disabled 命令，在 bridge disabled 中执行
        0x40 0x10 0x0010 0x80 0x20
    >;
    linka-init-cmd = <                                   # des linkA 初始化命令，在 bridge enable 中执行
        0x4c 0x10 0x06ff 0x11 0x0
        0x4c 0x10 0x01ce 0x4e 0x0
        0x4c 0x10 0x020c 0x03 0x0
        0x4c 0x10 0x020d 0xaa 0x0
        0x4c 0x10 0x020e 0x4a 0x0
        0x1a 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
    >;
    ser-status = <                                       # 检测 ser 状态，是读 i2c 命令，注意只支持如下 3 条命令并
        0x40 0x10 0x00d 0x80 0x80                       # 串行器是否能 detect 到
        0x40 0x10 0x102 0x80 0x80                       # 96789 检查 PCLK 是否 detect 到
        0x40 0x10 0x55d 0x70 0x70                       # 96789 检查 HS/Vs/DE 是否 detect 到
    >;
    linka-status = <                                     # 检测 linka 状态，注意只支持如下两条命令并且要按顺序
        0x40 0x10 0x01f 0x08 0x08                       # 96789 检查 linka 是否有接入
        0x4c 0x10 0x6ff 0x11 0x11                       # 96789 检查 linka 是否有被初始化，通过往一个用不到且能
        读写的 reg 里写任意值再读出来判断它是否被初始化
    >;
    panel-timing-a {                                    # panel timing 信息
        width = <1920>;
        height = <1080>;
        hfp = <40>;
        hsa = <42>;
        hbp = <78>;
        vfp = <24>;
        vsa = <3>;
        vbp = <9>;
        fps = <60>;
        width-mm = <129>;
        height-mm = <64>;
    };
};

```

10.4 DSI Superframe 设定示例

```

huayang_dsi_superframe: config2 {
    superframe = <1>;
    pre-init-cmd = <
        0x40 0x10 0x0002 0x73 0x00
        0x40 0x10 0x0053 0x10 0x00
        .....
    >;
    i2c-remap-cmd = <
        .....
    >;
    post-init-cmd = <>;
    linka-init-cmd = <
        0x48 0x10 0x01ce 0x4c 0x00
        .....
        0x1b 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0xd4
    >;
    linkb-init-cmd = <
        0x4a 0x10 0x01ce 0x4c 0x00
        .....
    >;
    linka-deinit-cmd = <
        0x1b 0x00 0x0b 0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
    >;
    linkb-deinit-cmd = <
        0x1c 0x00 0x0b 0x83 0x00 0x00 0x02 0x00 0x00 0x00 0x00 0x00 0x00 0x85
    >;
    deinit-cmd = <>;
    ser-status = <
        0x40 0x10 0x00d 0x80 0x80
        0x40 0x10 0x102 0x80 0x80
        0x40 0x10 0x55d 0x70 0x70
    >;
    linka-status = <
        0x40 0x10 0x01f 0x08 0x08
        0x48 0x10 0x6ff 0x11 0x11
    >;
    linkb-status = <
        0x40 0x10 0x01f 0x10 0x10
        0x4a 0x10 0x6ff 0x11 0x11
    >;
}

```

des panel 的 node

是否为 superframe 模式，是写 1，否写 0

ser 初始化命令，在 bridge pre_enable 中执行

i2c reg 映射设定，在 bridge pre_enable 中执行，plug in 后会重发

ser 初始化命令，在 bridge pre_enable 中执行

linka 初始化命令，在 bridge enable 中执行

linkb 初始化命令，在 bridge enable 中执行

linkA deinit 命令，在 bridge disable 中执行

linkB deinit 命令，在 bridge disable 中执行

ser deinit 命令，在 bridge disable 中执行，只有 linka/b 都 disable 了才会执行

检测 ser 状态，是读 i2c 命令，注意只支持如下 3 条命令并且要按顺序填写

串行器是否能 detect 到

96789 检查 PCLK 是否 detect 到

96789 检查 HS/Vs/DE 是否 detect 到

检测 linka 状态，注意只支持如下两条命令并且要按顺序

96789 检查 linka 是否有接入

96789 检查 linka 是否有被初始化，通过往一个用不到且能读写的 reg 里写任意值再读出来判断它是否被初始化

检测 linkb 状态，注意只支持如下两条命令并且要按顺序

96789 检查 linkb 是否有接入

96789 检查 linkb 是否有被初始化，通过往一个用不到且能读写的 reg 里写任意值再读出来判断它是否被初始化

```

panel-timing-a {                                # panel A timing, 必须设置
    width = <1920>;
    height = <1080>;
    hfp = <40>;
    hsa = <41>;
    hbp = <79>;
    vfp = <24>;
    vsa = <3>;
    vbp = <9>;
    fps = <60>;
    width-mm = <129>;
    height-mm = <64>;
};

panel-timing-b {                                # panel B timing, superframe 和 dual link 模式下必
须设置
    width = <1920>;
    height = <1080>;
    hfp = <40>;
    hsa = <41>;
    hbp = <79>;
    vfp = <24>;
    vsa = <3>;
    vbp = <9>;
    fps = <60>;
    width-mm = <129>;
    height-mm = <64>;
};
};

```

10.5 DSI Dual Link 设定示例

```

&i2c1 {
    status = "okay";
    max96789: max96789@40 {
        compatible = "maxim,max96789,dsi0";          # dsi0 对应的设定不变
        status = "disabled";
        reg = <0x40>;
        reset-gpios = <&pio 37 0>;
        pinctrl-names = "default";
        config = <&huayang_1080>;                    # 配置此时使用哪个 des panel 设置, 该 panel 设定
需要设置 dual-link=1
        port {
            max96789_in: endpoint {
                remote-endpoint = <&dsi_out>;          # 标准的 bridge node 写法
            };
        };
    };
    max96789_v0: max96789-v0@4a {                    # dual link 时虚拟的 driver
        compatible = "maxim,max96789,virtual";
        status = "disabled";
        reg = <0x4a>;
    };
};

```

```

master = <max96789>;                                # 必须设置，该设定对应到哪个实体设备
port {
    max96789_v0_in: endpoint {
        remote-endpoint = <&dsil_out>;                # 与 dsil 建立连接
    };
};
};
&dsil {
    status = "disabled";
    #address-cells = <1>;
    #size-cells = <0>;
    ports {
        port {
            dsil_out: endpoint {
                remote-endpoint = <&max96789_v0_in>;    # 与 max96789_v0 建立连接
            };
        };
    };
};
huayang_1080: config1 {                                # 对应 serdes panel 设定
    dual-link = <1>;                                    # 设置 dual-link mode
    pre-init-cmd = <
        0x40 0x10 0x0001 0x08 0x00
        0x40 0x10 0x0203 0x00 0x00
        .....
    >;
    panel-timing-a {                                    # panel A timing, 必须设置
        width = <1920>;
        height = <1080>;
        hfp = <40>;
        hsa = <41>;
        hbp = <79>;
        vfp = <24>;
        vsa = <3>;
        vbp = <9>;
        fps = <60>;
        width-mm = <129>;
        height-mm = <64>;
    };
    panel-timing-b {                                    # panel B timing, dual link 模式下必须设置
        width = <1920>;
        height = <1080>;
        hfp = <40>;
        hsa = <41>;
        hbp = <79>;
        vfp = <24>;
        vsa = <3>;
        vbp = <9>;
        fps = <60>;
        width-mm = <129>;
        height-mm = <64>;
    };
};

```

```
};
};
```

10.6 Des Panel 兼容设定示例

```
max96789: max96789@40 {
    compatible = "maxim,max96789,dsi0";      # ser 的设定
    status = "disabled";
    reg = <0x40>;
    reset-gpios = <&pio 37 0>;
    pinctrl-names = "default";
    config = <&config_compatible>;           # 使用 des panel 兼容设定
    .....
};
&max96789 {
    config_compatible: compatible-node {
        comp-cmd = <                                # 识别兼容设定前发送的命令，格式同前面提到的写命令
            0x1a 0x00 0x0b 0x83 0x50 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0xd4
        >;
        comp-type =                                # 读取兼容屏的设定
            <0x1a 0x08 0x08 0xf0 0x10 &config_bt>, # 格式同前面提到的读命令，最后一个元素为匹配到后
            对应的屏设定 handle
            <0x1a 0x08 0x08 0xff 0xff &huayang_1080>; # 如有其他兼容设定，按这个格式加在后面。如果
            都匹配不到，取最后一行的设定
    };
}
```

10.7 使用注意

- 以上各种 cmd 都是可选的，可以不设置或者写空
- 当 status 命令为空时，不会启动 hotplug 线程
- 只有单屏时，必须设置 panel-timing-a
- i2c-remap-cmd 会在 hotplug 的时候重新发送，需要在 hotplug 后重新发送的命令放在这里
- 命令发送顺序为：pre-init-cmd → i2c-remap-cmd → post-init-cmd → linka-init-cmd → linkb-init-cmd → linka-deinit-cmd → linkb-deinit-cmd → deinit-cmd
- 使用虚拟 driver 时，需要将 virtual_dsiX(for dsi, edp/dp is virtual_edp..) display_volX_exdmaX,max96789_vX 这几个同时 enable，或者同时 disabled

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿赔偿责任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。